



MI -SISTEMAS DIGITAIS

MARCO 3 - APLICAÇÃO C

DISCENTES:

MARCOS VINICIUS DIAS OLIVEIRA

MATHEUS SILVA RODRIGUES

DOCENTE:

ANFRANSERAI MORAIS DIAS

SUMÁRIO

- 1. INTRODUÇÃO
- 2. FUNDAMENTAÇÃO TEÓRICA
- 3. DESENVOLVIMENTO
- 4. MÓDULO VGA (vga.c)
- 5. MÓDULO MOUSE (mouse.c)
- 6. MÓDULO CSV (csv.c)
- 7. INTERFACE E MODOS DE OPERAÇÃO
- 8. MÉTRICAS E TESTES

INTRODUÇÃO

REQUISITOS DO MARCO:

- APLICAÇÃO EM C COM CLI (CLASSIFICAÇÃO INDIVIDUAL + DATASET + BENCHMARK)
- INTEGRAÇÃO COM O IP-CORE VGA PARA EXIBIR A IMAGEM CLASSIFICADA
- MODO DE DESENHO DE IMAGEM USANDO O MOUSE
- MODO DE VALIDAÇÃO/BENCHMARK RODANDO N IMAGENS
- CÁLCULO DE ACURÁCIA, LATÊNCIA, JITTER E THROUGHPUT
- SALVAR LOG EM CSV

ESTRUTURA DO PROJETO

- Makefile, driver.h, driver.s, main.c - mantidos do Marco 2
- src/vga.c - controle do IP-Core VGA (exibição da imagem)
- src/mouse.c - leitura do mouse via /dev/input/event0
- src/csv.c - geração do log de benchmark em CSV
- archives/ - pesos (W_{in} , b, beta) e imagens de teste (test/)

FUNDAMENTAÇÃO TEÓRICA

IP-CORE VGA

- Mapeado via MMIO no mesmo /dev/mem usado pelo coprocessador ELM
- Base física: 0xFF200000
- Offset do registrador de controle: 0x0030
- Resolução da tela: 320 x 240 pixels
- Cada pixel é escrito com um único registrador de 32 bits, contendo posição (x,y), cor (R,G,B de 3 bits) e um bit de escrita (write enable)

DESENVOLVIMENTO

MÓDULO VGA (vga.c)

- `vga_start()` mapeia o ponteiro de controle do VGA a partir do mesmo mmap do driver ELM (`elm_mmap()` + offset 0x0030)
- `vga_pixel(x, y, r, g, b)` monta um único valor de 32 bits combinando posição e cor via shifts e OR, e o envia em dois passos: escreve o valor com o bit de write ligado, depois desliga o bit
- Esse handshake (subir e descer o bit de escrita) é o mesmo padrão usado no envio de instruções ao coprocessador no Marco 2

MONTAGEM DO PIXEL

- O valor de 32 bits é montado assim:
- bits [8:0] → coordenada X (até 511)
- bits [16:9] → coordenada Y (até 255)
- bits [20:18] → componente R (3 bits, 0-7)
- bits [23:21] → componente G (3 bits, 0-7)
- bits [26:24] → componente B (3 bits, 0-7)
- bit [27] → write enable (1 = escreve o pixel)

EXIBIÇÃO DA IMAGEM (vga_draw)

- A imagem MNIST tem 28x28 pixels em escala de cinza (784 bytes), mas a tela é 320x240, então cada pixel da imagem é desenhado como um bloco de 7x7 pixels na tela
- A imagem é centralizada com um offset fixo: x começa em 62, y começa em 22
- Cada byte do pixel (0-255) é reduzido para 3 bits (0-7) com um shift à direita (`px >> 5`), já que o VGA só aceita 3 bits por canal de cor
- Resultado: uma imagem 196x196 (28x7) desenhada em tons de cinza no centro da tela

FUNÇÕES AUXILIARES DO VGA

- `vga_reset()` - varre toda a tela (320x240) pintando preto, para limpar o frame entre uma classificação e outra
- `vga_border()` - desenha uma borda nas áreas que ficam fora da imagem 196x196, dando contraste visual à área de desenho
- `vga_draw_cell()` / `vga_restore()` - usadas pelo modo de desenho com mouse, para redesenhar só a célula do pixel sob o cursor (sem precisar redesenhar a tela toda)

MÓDULO MOUSE (mouse.c)

- Lê eventos do dispositivo Linux `/dev/input/event0` diretamente, sem biblioteca gráfica
- Cada evento é uma struct `input_event`, com campos `type`, `code` e `value`
- Existem três tipos de evento tratados: `EV_REL` (movimento relativo), `EV_KEY` (clique de botão) e `EV_SYN` (sincronização, sinaliza que o pacote de eventos terminou)
- O loop principal só atualiza a tela quando recebe um `EV_SYN`, evitando redesenhar o cursor a cada pequeno evento

ACÚMULO DE MOVIMENTO E MODOS

- Eventos REL_X e REL_Y chegam seprados, então dx e dy são acumulados até o EV_SYN, quando são somados de uma vez à posição do cursor (mouse_x, mouse_y)
- Foi feito um clamp para não sair da área da imagem: x entre 62 e 251, y entre 22 e 211
- BTN_LEFT pressionado ativa modo = 1 (desenhar)
- BTN_RIGHT pressionado ativa modo = 2 (apagar)
- BTN_MIDDLE encerra o loop e confirma a imagem desenhada

DESENHO NO VETOR DE IMAGEM

- A cada novo EV_SYN: a célula antiga é restaurada com `vga_restore()` e o cursor é redesenhado na nova posição com `vga_draw_mouse()`
- A posição em pixels da tela é convertida para a célula 28x28 da imagem: $px = (mouse_x - 62) / 7$, $py = (mouse_y - 22) / 7$
- No modo desenhar (modo 1): o pixel central recebe valor 255 e os 4 vizinhos (cima, baixo, esquerda, direita) recebem 120
- No modo apagar (modo 2): o pixel sob o cursor é zerado

MÓDULO CSV (csv.c)

- Responsável por registrar o log do benchmark em um arquivo .csv, usado como evidência dos testes
- `create_csv()` monta o nome do arquivo com data e hora (`benchmark_AAAAMMDD_HHMMSS.csv`), usando `time()` e `localtime()`, e escreve o cabeçalho das colunas
- `infs_csv()` é chamada uma vez por inferência durante o benchmark, registrando: número da inferência, nome do arquivo da imagem, valor predito, valor esperado, resultado (Correta/Incorreta) e latência em nanossegundos
- `smr_csv()` é chamada uma única vez ao final, escrevendo o resumo: total de imagens, acertos, erros, acurácia, latência média, throughput e desvio padrão

INTERFACE E MODOS DE OPERAÇÃO

MENU PRINCIPAL (main.c)

- Ao iniciar, o programa abre o driver (elm_open), reseta o coprocessador (elm_reset), carrega os pesos (elm_load) e inicializa o VGA (vga_start + vga_reset)
- Um loop do-while apresenta um menu de texto com 4 opções e lê a escolha com scanf
- 1 - Inferência enviando imagem (a partir de arquivo)
- 2 - Inferência desenhando a imagem (com o mouse)
- 3 - Benchmark
- 4 - Sair (fecha o driver com elm_close)

MODO 1 — INFERÊNCIA POR ARQUIVO

- O caminho da imagem pode vir como argumento de linha de comando (`argv[1]`) ou ser digitado, se não for passado
- O arquivo `.bin` é aberto e lido com `fread`, esperando exatamente 784 bytes (28x28)
- A imagem é exibida na tela com `vga_draw(img)`
- O usuário informa o dígito esperado
- `elm_start(img)` envia a imagem ao coprocessador e `elm_result()` lê o dígito predito
- O resultado é comparado com o esperado e o acerto/erro é impresso

MODO 2 — INFERÊNCIA DESENHANDO

- O vetor da imagem (784 bytes) é zerado antes de começar
- A função `draw(img)`, do módulo `mouse.c`, é chamada e bloqueia o programa até o usuário desenhar e confirmar com o botão do meio
- A partir daí, o fluxo é igual ao modo 1: pede o dígito esperado, chama `elm_start(img)`, lê o resultado com `elm_result()` e compara

MODO 3 — BENCHMARK

- Abre a pasta test/ com opendir/readdir e conta o total de imagens antes de começar
- Extrai o dígito esperado do nome
- Para cada imagem: lê o arquivo, marca o tempo com clock_gettime antes e depois de elm_start(), calcula a latência, chama elm_result(), compara com o esperado e grava a linha no CSV com infs_csv()
- Ao final, calcula as métricas agregadas (acurácia, latência média, throughput, jitter), imprime na tela e grava o resumo no CSV com smr_csv()

MÉTRICAS E TESTES

ACURÁCIA

- Mede quantas inferências bateram com o dígito esperado
- Duas variáveis contam o resultado de cada imagem: ok (correta) e wrng (incorreta), incrementadas conforme `r == e`
- Imagens com erro de leitura (eimg) ou erro de inferência (einf) são descontadas do total, pois não fazem sentido entrar no cálculo
- $\text{Acurácia} = \text{ok} / (\text{total} - \text{eimg} - \text{einf}) * 100$

LATÊNCIA

- Mede o tempo de uma inferência completa, do envio da imagem até a leitura do resultado
- Usa `clock_gettime(CLOCK_MONOTONIC, ...)` antes e depois de `elm_start(img)`
- A diferença entre os timestamps (`tv_sec` e `tv_nsec`) é convertida para nanossegundos
- Esse tempo inclui todo o handshake do driver: polling, escrita das instruções e espera pelo status DONE
- A latência média do benchmark é a soma das latências individuais dividida pelo número de inferências válidas

JITTER (DESVIO PADRÃO)

- Mede o quanto a latência varia de uma inferência para outra
- Todas as latências individuais são guardadas em um array (lats[])
- Para cada uma, calcula-se a diferença em relação à média (lat) e eleva-se ao quadrado, somando tudo na variância
- Jitter = raiz quadrada de (variância / número de testes válidos)

THROUGHPUT

- Mede quantas inferências o sistema consegue processar por segundo
- A soma de todas as latências (em nanossegundos) é convertida para segundos
- $\text{Throughput} = \text{número de inferências válidas} / \text{tempo total em segundos}$

RESULTADO DO BENCHMARK

- Dataset de teste: 2000 imagens
- Acertos: 768 | Erros: 1232
- Acurácia: 38,40%
- Latência média: 10.112.016 ns
- Throughput: 98,89 inferências/s
- Desvio padrão (jitter): 8.374.023 ns
- Erros de leitura/inferência: 0